DREAMS · Guía final de defensa oral

Base contrastada: `main`, commit `60451cabe5ff9b78f97777d64120e5ff9a3238b2`. Revisión documental: 10/11/2026. Las ubicaciones se actualizan a la reorganización app/ de PR64; la lógica explicada conserva la base 60451ca, sin cambios funcionales ni nueva QA remota. No se detectaron cambios entre esa base de la guía adjunta y main al iniciar esta tarea.

Esta guía es material de estudio con asistencia de IA, contrastado con archivos reales. No acredita programación, participación, revisión o dominio de los integrantes. No garantiza las preguntas ni la aprobación. No modifica ni despliega la oficial.

Cómo estudiar

Primero ensayá la respuesta breve sin leer. Después abrí el archivo indicado, seguí una petición y explicá el proceso con tus palabras. Ante un error, corregí y repetí. No memorices afirmaciones de autoría.

Los tres audios orientan los temas: entrada del programa, carpetas, funciones, Node/JavaScript/Express, métodos/endpoints, login POST y seguridad. Se usan sólo esos temas, sin copiar la transcripción, identificar a su participante ni publicar audios privados. Los ejemplos de otro proyecto no son rutas de DREAMS.

Recorrido para ubicarte

Navegador (HTML/CSS/JS) → /api → Pages/proxy → Worker privado → Express
Express → Supabase Auth (identidad y sesiones)
Express → Supabase Data API/PostgreSQL (datos)

Entrada Node local: `pnpm start` → `app/server/server.js:start()` → Express → `listen` en puerto 8080 por defecto. Entrada del backend oficial: `app/worker/official.mjs:fetch_official()`; Cloudflare invoca fetch y el adaptador conecta con Express. Pages es la entrada pública/proxy; `index.html` es una página, no el arranque del backend.

Carpetas y archivos para mostrar

Carpeta/archivo	Función real
app/public/*.html	Pantallas públicas; catálogo, cuenta, carrito, checkout y demás páginas.
app/public/js/	Controladores del navegador. No guardan la clave de servidor.
app/public/css/style.css	Paletas, composición y responsive.
app/public/assets/	Recursos visuales compartidos.
app/server/server.js	Arranque Node local y configuración de servicios.
app/server/app.js	Construcción Express, instaladores de rutas, validaciones, sesión, permisos y errores.
app/server/customer-registration.js	Registro normal y perfil customer.
app/server/account-cart.js / reviews.js / presentation-checkout.js	Reglas de carrito, opiniones y simulación académica.
app/views/admin.html	Vista Admin servida después de autorización.
app/templates/	Footer compartido y plantilla gráfica social; sin Jinja.
app/pages/	Proxy, assets aprobados y SEO. No es la base de datos.

Carpeta/archivo	Función real
app/worker/official.mjs / official-client.mjs	Entrada del backend oficial y cliente Supabase.
app/supabase/migrations/	Historial SQL; no se reescribe lo aplicado.
app/supabase/rollback/	Recuperación compatible, no para ejecutar por rutina.
app/test/	Regresiones y fixtures; no equivalen a pruebas remotas nuevas.
app/scripts/	Builds, checks y mantenimiento. Herramientas históricas no se ejecutan por su nombre.
docs/	Guías, rúbrica, manual, wireframes, D.E.R., evidencia y registro de IA.
.github/ / app/package.json / app/pnpm-lock.yaml	CI, comandos y dependencias reproducibles. Railway/previews son antecedentes, no nuevos destinos autorizados.

Temas y repreguntas para el oral

01. ¿Qué es DREAMS?

Respuesta breve: DREAMS es un sitio de perfumería multimarca hecho con HTML, CSS y JavaScript. Tiene catálogo, filtros, cuentas, carrito, opiniones y un panel administrador. El backend usa Node.js/Express y Supabase; el checkout es académico.

Desde cero y ejemplo de DREAMS: No lo presentes como un sistema de cobros reales. La interfaz permite demostrar la compra, pero el flujo académico no inserta pedidos comerciales ni descuenta stock. La descripción tiene que coincidir con la versión que efectivamente mostrás.

Archivo y función: README.md; app/public/index.html; app/server/presentation-checkout.js.

Repregunta posible: ¿Qué parte corre en el navegador y cuál en el servidor?

Error frecuente: No presentar el checkout de demostración como un cobro comercial.

02. ¿Cuál es el archivo principal que corre el proyecto?

Respuesta breve: Para arrancar con Node, el comando pnpm start ejecuta app/server/server.js. Allí se leen variables de entorno, se crea el cliente de Supabase, se configura la aplicación Express y se escucha en un puerto. En el despliegue oficial de Cloudflare, la entrada es app/worker/official.mjs.

Desde cero y ejemplo de DREAMS: Hay dos contextos y no hay que confundirlos. app/package.json define start como node app/server/server.js; el puerto por defecto del arranque Node es 8080. app/server/app.js exporta la construcción de la aplicación, pero por sí solo no es el comando de arranque. Cloudflare invoca el handler fetch del Worker oficial y este adapta Express con handleAsNodeRequest; Pages hace de entrada pública/proxy. No digas que todo se ejecuta desde index.html.

Archivo y función: app/package.json; app/server/server.js; app/server/app.js; app/worker/official.mjs; app/pages/proxy.mjs.

Repregunta posible: ¿Qué ejecuta pnpm start y qué ejecuta Cloudflare?

Error frecuente: index.html no arranca el backend; app/server/app.js construye la app y no es por sí solo el comando de inicio.

03. ¿Cómo están organizadas las carpetas?

Respuesta breve: public contiene las páginas y los archivos que usa el navegador; server contiene lógica de backend; views incluye el HTML privado del Admin; supabase guarda esquema y migraciones; worker y pages corresponden al despliegue; test y scripts reúnen pruebas y herramientas; docs contiene la documentación.

Desde cero y ejemplo de DREAMS: app/public/js/cuenta.js maneja el formulario en el navegador; app/server/customer-registration.js valida y procesa el registro en el servidor. El HTML del Admin está en app/views/admin.html para servirse después de la autorización. Estar en una carpeta privada no sustituye el middleware que controla el acceso. Los secretos no deben estar en public ni versionados.

Archivo y función: app/public/; app/server/; app/views/admin.html; app/supabase/; app/worker/; app/pages/; app/test/; app/scripts/; docs/.

Repregunta posible: ¿Por qué admin.html está en views y no basta con esconder su enlace?

Error frecuente: La carpeta organiza archivos; el permiso se comprueba en el servidor.

04. ¿Qué hace cada lenguaje?

Respuesta breve: HTML define la estructura y el contenido, CSS la presentación y adaptación a pantallas, y JavaScript la interacción y las peticiones. En DREAMS hay JavaScript tanto en navegador como en servidor.

Desde cero y ejemplo de DREAMS: En app/public/index.html están header, nav, main, secciones y footer; app/public/css/style.css determina aspecto y layout. app/public/js/app.js carga productos y actualiza la interfaz. El JavaScript de server usa APIs de Node y no debe enviarse como código cliente con secretos. Compartir lenguaje no significa compartir permisos.

Archivo y función: app/public/index.html; app/public/css/style.css; app/public/js/app.js; app/server/app.js.

Repregunta posible: ¿Qué diferencia hay entre HTML semántico y estilos CSS?

Error frecuente: HTML estructura; CSS presenta; JavaScript agrega lógica. No son intercambiables.

05. ¿Qué es JavaScript y dónde lo usás?

Respuesta breve: Es un lenguaje de programación. En el navegador se usa para eventos, formularios, filtros, carrito y contenido dinámico. En el backend, con Node.js, se usa para rutas, validaciones y acceso a los servicios.

Desde cero y ejemplo de DREAMS: Un evento como submit captura el envío de un formulario. fetch envía una petición HTTP. Una respuesta JSON se transforma en datos que la interfaz utiliza. DOMContentLoaded permite inicializar cuando el documento está listo. JavaScript no es Java.

Archivo y función: app/public/js/cuenta.js; app/public/js/catalogo.js; app/public/js/app.js.

Repregunta posible: ¿Un mismo archivo JavaScript puede ejecutarse sin cambios en cualquier entorno?

Error frecuente: El DOM es del navegador; process y otros módulos pertenecen al entorno Node.

06. ¿Qué es Node.js?

Respuesta breve: Es un entorno de ejecución de JavaScript fuera del navegador. En DREAMS permite ejecutar el backend, usar módulos, configurar Express y comunicarse con Supabase.

Desde cero y ejemplo de DREAMS: Node no es el lenguaje ni la base de datos ni Express. app/package.json pide Node 24.x para reproducir este proyecto. require importa módulos CommonJS; los archivos .mjs usan

módulos ESM. Las operaciones de red son asíncronas. No hace falta ejecutar un servidor Node con listen en tu computadora para visitar la oficial ya desplegada.

Archivo y función: app/package.json; app/server/server.js; app/worker/official.mjs.

Repregunta posible: ¿Node necesita una pestaña de navegador para atender peticiones?

Error frecuente: Node ejecuta JavaScript; no es una biblioteca de interfaz ni una base de datos.

07. ¿Qué es Express y por qué está en DREAMS?

Respuesta breve: Express es un framework web para Node.js. Permite definir rutas HTTP, encadenar middlewares y manejar request y response. En DREAMS organiza la API y sus controles de validación, sesión y permisos.

Desde cero y ejemplo de DREAMS: Express se instala como dependencia y se importa en app/server/app.js. Una ruta como GET /api/products obtiene productos; un middleware puede verificar la sesión antes de llegar al handler. Una justificación técnica es que simplifica esas tareas sin programarlas desde cero con node:http. Eso no acredita por sí solo quién eligió la tecnología: explicá sus ventajas sin inventar una decisión personal.

Archivo y función: app/package.json; app/server/app.js: require(express), configure_router e install_get_api_products.

Repregunta posible: ¿Podría hacerse sin Express?

Error frecuente: Express es un framework web para Node, no un lenguaje ni una base de datos.

08. ¿Cuál es la diferencia entre URL, ruta, endpoint y método?

Respuesta breve: La URL es la dirección completa. La ruta es su parte de ubicación, como /api/products. El endpoint es una operación de la API identificada por método y ruta. GET /api/products y POST /api/admin/products son operaciones diferentes.

Desde cero y ejemplo de DREAMS: https://dreams-perfumes.pages.dev/catalogo.html es una página; GET /api/products devuelve datos. En /api/products/12, el 12 es un parámetro de ruta. En /api/products?limit=8, limit es un parámetro de consulta. localhost significa tu propia computadora; no es el dominio de la página pública. El ejemplo /api/productos del audio no es el nombre real de la ruta de DREAMS.

Archivo y función: app/server/app.js; app/public/catalogo.html; app/pages/proxy.mjs.

Repregunta posible: ¿/producto.html y /api/products/:id devuelven lo mismo?

Error frecuente: La página devuelve HTML; el endpoint de datos devuelve JSON. El método también define la operación.

09. ¿Qué hacen GET, POST, PUT, PATCH y DELETE?

Respuesta breve: GET consulta; POST envía datos para crear o procesar una operación; PUT actualiza una representación; PATCH suele modificar parcialmente; DELETE elimina. En DREAMS se usan GET, POST, PUT y DELETE; no hay que afirmar que PATCH está implementado por conocer la definición.

Desde cero y ejemplo de DREAMS: GET /api/products lista; POST /api/auth/login procesa credenciales; POST /api/admin/products crea; PUT /api/admin/products/:id actualiza; DELETE /api/admin/products/:id elimina. Una ruta GET no debería efectuar una modificación de negocio. El método no vuelve una operación segura por sí mismo: hacen falta HTTPS, validaciones y permisos.

Archivo y función: app/server/app.js; app/server/official-policy.js.

Repregunta posible: ¿PUT y DELETE están protegidos sólo por su nombre?

Error frecuente: El método indica intención; la validación y los permisos del servidor impiden operaciones indebidas.

10. ¿Por qué el login usa POST y no GET?

Respuesta breve: Porque envía correo y contraseña en el cuerpo de la solicitud para procesar una autenticación. No conviene incluir contraseñas en parámetros de URL, donde podrían quedar en historial o registros.

Desde cero y ejemplo de DREAMS: POST no cifra el contenido: HTTPS protege el transporte. El servidor debe evitar logs del cuerpo con contraseñas y validar las entradas. En DREAMS, POST /api/auth/login llama a signInWithPassword y, si la autenticación es válida, establece cookies de sesión.

Archivo y función: app/public/js/cuenta.js; app/server/app.js: install_post_api_auth_login; app/server/auth-cookies.js.

Repregunta posible: ¿POST cifra el cuerpo o evita que el servidor lo registre?

Error frecuente: POST no cifra: HTTPS protege el transporte y deben evitarse logs de credenciales.

11. ¿Qué API consume DREAMS?

Respuesta breve: El navegador consume la API propia /api del mismo sitio. El backend usa el SDK de Supabase para comunicarse con Supabase Auth y su Data API sobre PostgreSQL. Esos son los servicios externos utilizados.

Desde cero y ejemplo de DREAMS: WhatsApp abre un enlace wa.me con un mensaje preparado; no hay que describirlo como una API de mensajería integrada. Mercado Pago tiene código histórico/preparado, pero no se debe decir que procesa los cobros de la entrega. Una biblioteca cliente como supabase-js facilita llamadas a servicios; no es en sí toda la base de datos.

Archivo y función: docs/entrega-2026/api.md; app/worker/official-client.mjs; app/server/app.js; app/public/js/app.js.

Repregunta posible: ¿WhatsApp es una API consumida por DREAMS?

Error frecuente: El enlace de WhatsApp abre un servicio externo; no equivale a integrar una API de mensajería.

12. ¿Qué ocurre al abrir el catálogo?

Respuesta breve: El navegador carga HTML, CSS y JavaScript. El JavaScript solicita productos a /api. Pages/proxy y el backend procesan la solicitud; el servidor consulta Supabase, devuelve JSON y el navegador crea las tarjetas.

Desde cero y ejemplo de DREAMS: La arquitectura tiene lógica de catálogo también en Pages para los filtros; no todo se reduce a una única función Express. Para una explicación inicial alcanza el recorrido navegador → API → datos → JSON → interfaz; si preguntan por el despliegue, mostrás el proxy y las rutas que realmente intervienen. No se envía la Secret key al navegador.

Archivo y función: app/public/js/catalogo.js; app/public/js/app.js; app/pages/proxy.mjs; app/server/app.js; app/worker/official-client.mjs.

Repregunta posible: ¿Cómo se actualiza la pantalla cuando termina fetch?

Error frecuente: fetch no dibuja tarjetas por sí solo: el código procesa JSON y actualiza el DOM.

13. ¿Qué es una función y por qué hay varias?

Respuesta breve: Una función agrupa una tarea con un nombre, puede recibir parámetros y devolver un resultado. Separar responsabilidades evita repetir lógica y facilita entender y probar el código.

Desde cero y ejemplo de DREAMS: No alcanza con decir que hay muchas para cumplir la rúbrica. Debés poder explicar qué entra, qué sale, quién la llama, qué modifica y qué pasa si falla. El límite de 40 líneas es un criterio académico; dividir sin criterio en funciones difíciles de seguir no mejora el diseño.

Archivo y función: app/scripts/function-inventory.mjs; docs/entrega-2025/defensa.md.

Repregunta posible: ¿Una función que devuelve una Promise ya terminó su trabajo?

Error frecuente: Llamar una función async no garantiza éxito: hay que esperar y manejar rechazo.

14. ¿Qué hace normalize_search?

Respuesta breve: Prepara un texto para buscar sin distinguir mayúsculas, tildes ni espacios en los extremos. Convierte valores ausentes en una cadena vacía y devuelve el texto normalizado.

Desde cero y ejemplo de DREAMS: String(value ?? "") evita tratar null como texto de búsqueda. normalize("NFD") separa letras y marcas; replace elimina esas marcas; toLowerCase unifica mayúsculas y trim limpia extremos. No modifica productos ni la base. Ejemplo didáctico: " CHANÉL " pasa a "chanel".

Archivo y función: app/public/js/admin-product-search.js: normalize_search.

Repregunta posible: ¿Qué pasa con un valor null o un nombre con tilde?

Error frecuente: Normalizar facilita comparar textos; no modifica el nombre guardado ni evita por sí solo una inyección SQL.

15. ¿Qué hace filter_products?

Respuesta breve: Filtra la lista del Admin por nombre o marca y, si se pide, por disponibilidad baja. Devuelve una lista filtrada sin borrar productos.

Desde cero y ejemplo de DREAMS: filter conserva elementos cuyo predicado es verdadero; some comprueba si coincide al menos uno de los campos nombre/marca; includes busca una parte del texto. Combina coincidencia de texto con low_stock cuando low_only está activo. Los productos que desaparecen de pantalla no se eliminan de la base.

Archivo y función: app/public/js/admin-product-search.js: filter_products.

Repregunta posible: ¿filter modifica la lista original? ¿Qué diferencia hay con map y some?

Error frecuente: Filtrar la pantalla no borra filas de la base.

16. ¿Cómo identificás disponibilidad baja?

Respuesta breve: Primero se comprueba que stock sea un número entero seguro, no negativo. Después se considera bajo si es menor o igual a 2: incluye cero, uno y dos.

Desde cero y ejemplo de DREAMS: valid_stock rechaza null, strings y valores inválidos. low_stock combina esa validación con <= 2. stock_label distingue sin stock, stock bajo, disponible y stock no válido. Number(null) daría cero, por eso convertir antes de validar puede inventar disponibilidad.

Archivo y función: app/public/js/admin-product-search.js: valid_stock, low_stock y stock_label.

Repregunta posible: ¿Por qué null o un texto no se toman como stock cero?

Error frecuente: Disponibilidad baja exige un número entero válido no negativo: sólo 0, 1 o 2.

17. ¿Qué hace load_featured_products?

Respuesta breve: Busca el contenedor de destacados, muestra una espera, pide hasta ocho productos, valida la respuesta, evita repetir IDs, crea las tarjetas y prepara el carrusel. Si falla, muestra un mensaje.

Desde cero y ejemplo de DREAMS: fetch devuelve una promesa; await espera la respuesta dentro de una función async. response.json convierte el cuerpo JSON. Se valida response.ok y que sea un array. filter/findIndex quita IDs repetidos y forEach agrega tarjetas al DOM. El código actual limita a ocho, pero no afirma que esos ocho sean todos los productos de la tienda.

Archivo y función: app/public/js/app.js: load_featured_products; app/public/js/featured-carousel.js.

Repregunta posible: ¿Qué productos se muestran y por qué el carrusel no avanza solo?

Error frecuente: Se usan productos reales distintos; la navegación es manual, sin autoplay.

18. ¿Qué significan async, await, Promise y try/catch?

Respuesta breve: Una promesa representa una operación que terminará después. async permite devolver una promesa y usar await para esperar su resultado. try/catch permite manejar errores sin dejar la interfaz en un estado engañoso.

Desde cero y ejemplo de DREAMS: await no convierte toda la aplicación en bloqueante ni vuelve inmediato Internet. En load_featured_products se espera fetch y la lectura JSON; si la red o la validación fallan, catch informa el problema. Se deben distinguir error de transporte, respuesta HTTP y validación de negocio.

Archivo y función: app/public/js/app.js: load_featured_products; app/server/app.js.

Repregunta posible: ¿try/catch captura una Promise rechazada si no se espera?

Error frecuente: Un catch sin esperar ni devolver la Promise puede no capturar su rechazo.

19. ¿Qué son request, response, next y middleware?

Respuesta breve: request contiene datos de la solicitud; response permite devolver estado, headers y cuerpo; next pasa al siguiente paso. Un middleware procesa la petición antes o entre handlers.

Desde cero y ejemplo de DREAMS: En DREAMS se restauran sesiones y después se verifica el rol. configure_router define login y admin. login usa next si existe request.user y responde 401 si falta; admin permite solo request.user.is_admin y responde 403 en caso contrario. El orden de middleware importa.

Archivo y función: app/server/app.js: install_session_middleware y configure_router.

Repregunta posible: ¿Qué ocurre si un middleware no responde ni llama next?

Error frecuente: La petición puede quedar sin finalizar; next no envía una respuesta por sí mismo.

20. ¿Qué medidas de seguridad usarías al publicar?

Respuesta breve: HTTPS, secretos solo en servidor, validación de entradas, sesiones seguras, permisos por rol, políticas de datos, límites de solicitudes y manejo de errores sin filtrar información. También probaría que un cliente no pueda modificar datos de Admin.

Desde cero y ejemplo de DREAMS: Nombrar medidas no equivale a demostrar una auditoría integral. En el código hay cookies HttpOnly/SameSite/Secure, middleware de origen, límites de Auth y comprobación de perfil. Se deben mantener dependencias, configuración, backups y controles de acceso. Los tests locales no acreditan por sí solos producción.

Archivo y función: app/server/auth-cookies.js; app/server/auth-limits.js; app/server/app.js; app/server/official-policy.js; app/supabase/migrations/.

Repregunta posible: ¿Alcanza con validar el formulario del navegador?

Error frecuente: El navegador se puede modificar: el servidor vuelve a validar y comprobar permisos.

21. ¿Qué es hashing? ¿DREAMS lo usa?

Respuesta breve: Un hash de contraseña es una derivación de una sola vía: no se guarda la contraseña original para recuperarla. DREAMS usa Supabase Auth, que según su documentación almacena hashes bcrypt con salt aleatorio.

Desde cero y ejemplo de DREAMS: No digas que escribiste una función bcrypt en DREAMS: la responsabilidad es de Supabase Auth. Un salt evita que dos contraseñas iguales produzcan necesariamente el mismo hash; bcrypt incorpora un costo de cálculo. En el login el proveedor verifica la contraseña frente al hash almacenado. Este material no consulta hashes reales ni fija un costo bcrypt que no se haya verificado.

Archivo y función: app/server/customer-registration.js; app/server/app.js: signInWithPassword; https://supabase.com/docs/guides/auth/password-security.

Repregunta posible: ¿Qué aporta el salt y dónde se usa bcrypt?

Error frecuente: El repositorio no implementa bcrypt; lo gestiona Supabase Auth. No se consultaron hashes reales.

22. ¿Hash, cifrado y HTTPS son lo mismo?

Respuesta breve: No. El hash de contraseña es de una sola vía; el cifrado permite recuperar el contenido con la clave apropiada; HTTPS protege la comunicación. Necesitás protección de transporte además de almacenamiento seguro.

Desde cero y ejemplo de DREAMS: Base64 es codificación, no cifrado ni hash de contraseña. Un JWT firmado no debe tratarse como contenido secreto por estar codificado. Los tickets HMAC de DREAMS sirven para integridad/autenticidad, no para esconder toda la información ni para almacenar contraseñas.

Archivo y función: app/server/account-cart.js; app/server/presentation-checkout.js; app/server/auth-cookies.js.

Repregunta posible: ¿Un JWT firmado o Base64 esconden su contenido?

Error frecuente: Codificación, firma, hash y cifrado son cosas distintas; firmar no oculta.

23. ¿Qué es CORS y qué hace DREAMS?

Respuesta breve: CORS es un mecanismo del navegador para controlar qué respuestas puede leer una página desde otro origen. DREAMS usa normalmente /api en el mismo origen; además verifica el origen de solicitudes que modifican datos.

Desde cero y ejemplo de DREAMS: No aparece la dependencia cors en app/package.json. Hay un mutation_origin_guard que comprueba Origin/Referer y otras señales en escrituras, además de la allowlist oficial. No es lo mismo que habilitar Access-Control-Allow-Origin para compartir respuestas. CORS no autentica usuarios, no sustituye permisos y no impide a un cliente externo llamar a una API. Las cookies y la protección frente a solicitudes de otros sitios requieren atención propia.

Archivo y función: app/package.json; app/server/app.js: mutation_origin_guard; app/pages/proxy.mjs; app/server/official-policy.js.

Repregunta posible: ¿Una herramienta externa puede llamar a la API aunque el navegador aplique CORS?

Error frecuente: CORS no es autenticación ni un bloqueo general del servidor. No hay dependencia cors ni configuración propia de Access-Control-Allow-Origin en los módulos revisados.

24. ¿Autenticación y autorización son lo mismo?

Respuesta breve: Autenticación comprueba quién sos; autorización decide qué podés hacer. Supabase verifica el acceso y DREAMS consulta el perfil y su rol para permitir acciones Admin.

Desde cero y ejemplo de DREAMS: La cuenta se identifica por un UUID, no por el texto del nombre ni por un correo escrito en el formulario. customer_registration rechaza campos como role o is_admin y crea perfil customer. Ocultar un botón no protege el endpoint: el servidor debe rechazar la operación y las políticas de datos deben acompañar.

Archivo y función: app/server/customer-registration.js; app/server/app.js: profile_loader y configure_router.

Repregunta posible: ¿Un usuario autenticado puede entrar siempre al Admin?

Error frecuente: Tener sesión no concede el rol administrador ni el permiso sobre cualquier recurso.

25. ¿Cómo se mantiene la sesión?

Respuesta breve: El servidor crea cookies de acceso y renovación. En nuevas solicitudes comprueba el usuario y, si corresponde, renueva la sesión. Al salir intenta revocar la sesión actual y vence las cookies; no hay que prometer que todos los tokens dejan de servir inmediatamente.

Desde cero y ejemplo de DREAMS: auth-cookies.js configura HttpOnly, SameSite=Lax y Secure en producción. HttpOnly impide leer la cookie desde JavaScript, pero no elimina todos los riesgos de XSS ni de solicitudes inducidas. El middleware usa getUser y refreshSession, después consulta el perfil. No debe confiarse en un valor is_admin guardado por el navegador.

Archivo y función: app/server/auth-cookies.js; app/server/app.js: install_session_middleware, logout y clear_session.

Repregunta posible: ¿Un 200 de logout garantiza que todo JWT quedó invalidado?

Error frecuente: No: signOut local puede fallar y se registra un aviso; el handler igualmente borra cookies y devuelve 200. No elimina la cuenta.

26. ¿Qué significan 200, 201, 400, 401, 403, 404, 409, 429 y 503?

Respuesta breve: 200 es éxito; 201 creación; 400 solicitud inválida; 401 falta autenticación válida; 403 acceso prohibido; 404 no encontrado; 409 conflicto; 429 exceso de solicitudes; 503 indisponibilidad temporal.

Desde cero y ejemplo de DREAMS: El significado general no garantiza que todos los endpoints devuelvan siempre el código ideal. En DREAMS, el middleware login devuelve 401 sin usuario y el guard admin devuelve 403 si no es administrador, incluso si es anónimo. Los errores concretos deben leerse junto con el cuerpo y la ruta, no adivinarse por el número solamente.

Archivo y función: app/server/app.js: configure_router; app/server/account-cart.js; app/server/auth-limits.js.

Repregunta posible: ¿Qué diferencia hay entre credenciales inválidas y proveedor caído?

Error frecuente: Un error 503 no demuestra una contraseña incorrecta; no ocultar fallos de infraestructura como 401.

27. ¿Qué base de datos utiliza DREAMS?

Respuesta breve: Utiliza PostgreSQL mediante Supabase. La autenticación es un servicio de Supabase y las tablas de negocio incluyen productos, perfiles, opiniones y carritos, entre otras. PostgreSQL no es SQLite.

Desde cero y ejemplo de DREAMS: La documentación del D.E.R. identifica nueve tablas públicas y auth.users.id como frontera de Auth. No copies a producción un schema histórico sin revisar migraciones: el baseline usa brand y la tabla remota documentada usa marca; el contrato HTTP expone brand. No se verificó la base remota de nuevo para este Word.

Archivo y función: docs/auditoria-2025/base-de-datos.md; app/supabase/migrations/; app/server/product-schema.js.

Repregunta posible: ¿Supabase y PostgreSQL son lo mismo?

Error frecuente: Supabase es una plataforma de servicios; PostgreSQL es el motor relacional utilizado.

28. ¿Qué son PK, FK y cardinalidad?

Respuesta breve: PK identifica una fila de forma única. FK vincula un campo con una clave de otra tabla. La cardinalidad indica cuántas entidades pueden relacionarse, por ejemplo un usuario con muchas opiniones.

Desde cero y ejemplo de DREAMS: profiles.id es PK y FK a auth.users.id: cada perfil tiene un usuario Auth y un usuario puede tener cero o un perfil. reviews.user_id se relaciona con profiles. La relación de products con los artículos dentro de JSONB del carrito es lógica, no una FK por cada artículo; no dibujes una restricción inexistente. favorites y account_cart_operations tienen claves compuestas.

Archivo y función: docs/auditoria-2025/base-de-datos.md; docs/auditoria-2025/graficos/der.pdf.

Repregunta posible: ¿Los IDs dentro de items JSONB crean automáticamente una FK?

Error frecuente: Una asociación lógica dentro de JSON no implica una restricción FOREIGN KEY.

29. ¿Qué es RLS?

Respuesta breve: Row Level Security son políticas de PostgreSQL que limitan qué filas puede consultar o modificar un rol o usuario. Complementan los permisos del backend.

Desde cero y ejemplo de DREAMS: DREAMS debe verificar la sesión y el rol en el servidor y utilizar el JWT donde corresponde para que las políticas se apliquen. Una clave privilegiada puede saltar políticas; por eso se

conserva solo en servidor y no se usa indiscriminadamente. RLS no se demuestra ocultando botones ni por la sola existencia de una tabla.

Archivo y función: app/supabase/migrations/; app/server/app.js: admin_database; app/server/reviews.js.

Repregunta posible: ¿La credencial privilegiada obedece siempre las mismas políticas que un customer?

Error frecuente: Una clave privilegiada puede omitir RLS; por eso hay controles en backend y clientes con JWT donde corresponde.

30. ¿Cómo queda vinculado el carrito a un usuario?

Respuesta breve: El servidor toma el UUID de la sesión y guarda operaciones sobre su carrito. Usa identificadores de operación para evitar duplicaciones y recupera precios actuales desde productos. El visitante conserva una selección local.

Desde cero y ejemplo de DREAMS: GET y POST /api/cart/account requieren login. AccountCartService valida un contexto firmado que corresponde a la cuenta y llama a la RPC dreams_cart_operation con un operation_id. La selección guardada contiene IDs y cantidades; hydrate reconstruye información desde catálogo. Al cambiar de cuenta hay que evitar aplicar una operación pendiente de A a B.

Archivo y función: app/server/account-cart.js; app/public/js/account-cart.js; app/public/js/app.js; app/supabase/migrations/.

Repregunta posible: ¿Qué pasa si cambia la sesión mientras hay una operación pendiente?

Error frecuente: El servidor deriva el dueño de la sesión y verifica el contexto; no confía en un UUID enviado como autoridad.

31. ¿Cómo evitás que alguien borre una opinión ajena?

Respuesta breve: El autor se obtiene de la sesión. Para eliminar se comprueba el login y se filtra por id de opinión y user_id del usuario, usando un cliente con su JWT. La interfaz muestra la acción solo cuando corresponde, pero la protección decisiva está en servidor y políticas.

Desde cero y ejemplo de DREAMS: delete_own_review no acepta que el navegador elija otro propietario. Si la consulta no devuelve una opinión propia, responde que no se encontró. El administrador conserva una ruta de moderación diferente. operation_id también ayuda a evitar duplicados al publicar opiniones.

Archivo y función: app/server/reviews.js: publish_review, replay_review y delete_own_review.

Repregunta posible: ¿Basta con ocultar el botón Eliminar de otra opinión?

Error frecuente: La API verifica usuario y fila; ocultar un control no evita peticiones manuales.

32. ¿Cómo sabés si un cambio del Admin quedó guardado?

Respuesta breve: No alcanza con cambiar el HTML ni con ver un mensaje. El servidor debe validar, autorizar y escribir; después la aplicación verifica que los campos persistieron. Recargar y consultar otra vez permite comprobar el resultado.

Desde cero y ejemplo de DREAMS: Si se pierde la respuesta de un alta, repetirla sin comprobar podría crear un duplicado. Las validaciones de imagen en navegador ayudan a la experiencia, pero el backend también valida los datos y la autorización. Las pruebas de escritura deben hacerse sobre un producto de prueba autorizado, no sobre stock comercial.

Archivo y función: app/public/js/admin.js; app/public/js/admin-image-url.js; app/server/app.js: rutas /api/admin/products.

Repregunta posible: ¿Qué pasa si la escritura se hizo pero se perdió su respuesta?

Error frecuente: No hay que repetir un alta a ciegas ni mostrar éxito sin readback; existe recuperación de altas inciertas.

33. ¿Dónde hay condicionales, ciclos y operadores lógicos?

Respuesta breve: Hay if para validar, for y forEach para recorrer datos, filter/map/some para transformar o seleccionar y operadores como && y || para combinar condiciones. Puedo mostrar ejemplos del buscador y el carrito.

Desde cero y ejemplo de DREAMS: En low_stock, valid_stock(value) && value <= 2 exige ambas condiciones. filter_products usa coincidencia en nombre O marca y además aplica stock bajo cuando corresponde. Un return devuelve un valor o termina una rama; un throw produce un error para manejarlo en otro nivel. Explicá valores falsy y ?? sin afirmar que || y ?? son idénticos.

Archivo y función: app/public/js/admin-product-search.js; app/server/account-cart.js; app/server/app.js: normalize_cart_items.

Repregunta posible: ¿Qué diferencia hay entre filter, some y map?

Error frecuente: filter selecciona; some responde si hay coincidencia; map transforma. No escriben la base por sí solos.

34. ¿Cómo se adapta y cómo se navega?

Respuesta breve: HTML usa elementos como header, nav, main, section y footer; CSS adapta la distribución a distintos tamaños. El menú, los filtros y el carrusel tienen controles de teclado y etiquetas accesibles.

Desde cero y ejemplo de DREAMS: Responsive no significa simplemente achicar todo. Hay que conservar lectura, controles táctiles y ausencia de desbordes. featured-carousel.js responde a teclado y prefiere movimiento inmediato cuando el usuario solicita reducir animaciones. Su indicador puede existir solo para lectores de pantalla aunque no se vea.

Archivo y función: app/public/index.html; app/public/css/style.css; app/public/js/featured-carousel.js; app/public/js/navigation.js.

Repregunta posible: ¿Cómo se usa el carrusel sin mouse o con movimiento reducido?

Error frecuente: Responsive no significa sólo achicar texto; hay que mantener controles, foco y lectura.

35. ¿Para qué sirven Git, GitHub, tests y build?

Respuesta breve: Git registra versiones; GitHub aloja el repositorio y facilita revisión. Los tests comprueban casos y regresiones. El build prepara el bundle o archivos de publicación. Son pasos distintos de desplegar.

Desde cero y ejemplo de DREAMS: Un test con fixtures puede pasar aunque falte una clave de producción. No confundas CI verde, push y sitio actualizado. app/package.json define check, test, build:pages y build:official. Hay archivos de evidencia histórica; este documento no volvió a ejecutar la suite ni auditó el runtime oficial.

Archivo y función: app/package.json; .github/; app/scripts/; docs/evidence/.

Repregunta posible: ¿Un build aprobado demuestra que login remoto funciona?

Error frecuente: El build genera recursos; no equivale a una prueba remota de Auth, Admin ni despliegue.

36. ¿Qué harías si el profesor elige una función que no esperabas?

Respuesta breve: Primero ubico si corre en navegador o servidor. Leo nombre, parámetros, validaciones, llamadas, retorno y efectos. La relaciono con la pantalla y explico un caso normal y uno de error.

Desde cero y ejemplo de DREAMS: No inventes. Si algo no está claro, explicá lo que sí podés comprobar y seguí el flujo de llamadas. Practicá en una copia local: cambiar un texto, ajustar una validación y ejecutar un test. No toques credenciales, datos ni deployment durante la defensa para improvisar una demostración.

Archivo y función: docs/entrega-2025/defensa.md.

Repregunta posible: ¿Cómo ubicarías entradas, retorno y errores sin memorizar el nombre?

Error frecuente: Explicar una función exige seguir su proceso y efectos; no inventar una intención que no se deduce del código.

37. ¿Qué es un decorador y dónde hay algo parecido?

Respuesta breve: Un decorador envuelve una función para agregar comportamiento. DREAMS no usa @app.route de Flask; usa rutas Express, middleware y un envoltorio de errores llamado async_route.

Desde cero y ejemplo de DREAMS: En JavaScript una función puede recibir otra y devolver una nueva. async_route devuelve un handler que ejecuta el original y deriva errores con catch(next). login y admin comprueban permisos; route registra las rutas. Es útil distinguir el patrón de envoltorio de la sintaxis de decoradores, que no usamos aquí.

Archivo y función: app/server/app.js: application_context (async_route), configure_router (login, admin, route).

Repregunta posible: ¿Qué pasa si un handler async rechaza su Promise?

Error frecuente: No llamar decorador de Flask a cualquier middleware ni inventar una función @app.route.

38. ¿Qué son los templates de DREAMS?

Respuesta breve: Son estructuras reutilizables. El footer compartido está en app/templates/public-footer.html y se incorpora al preparar las páginas. No usamos Jinja ni un motor de templates de Flask.

Desde cero y ejemplo de DREAMS: La plantilla evita repetir a mano el mismo footer. app/scripts/sync-public-footer.mjs reemplaza ese fragmento en HTML y build-pages.mjs también lo incorpora. app/templates/social-card.html es fuente gráfica; app/views/admin.html es la vista HTML protegida. Una carpeta no protege por sí sola: lo hace la autorización del servidor.

Archivo y función: app/templates/public-footer.html; app/templates/social-card.html; app/scripts/sync-public-footer.mjs; app/scripts/build-pages.mjs; app/views/admin.html.

Repregunta posible: ¿Se procesa el footer en cada petición o durante la preparación de los recursos?

Error frecuente: No afirmar que todos los archivos .html son templates de servidor ni que hay Jinja.

39. ¿Qué hace el event loop de Node?

Respuesta breve: Permite coordinar tareas y atender otros eventos mientras se esperan operaciones como una consulta de red. async y await ayudan a escribir ese flujo.

Desde cero y ejemplo de DREAMS: Node ejecuta el JavaScript del hilo principal y delega distintas operaciones al entorno. Una Promise representa un resultado futuro; await suspende esa función hasta resolverlo, no todo el servidor. Una tarea de CPU larga y sin ceder puede bloquear el hilo. No es correcto resumirlo como que todo corre en paralelo.

Archivo y función: app/server/app.js: handlers async; app/server/server.js: start; app/public/js/catalogo.js: load_catalog.

Repregunta posible: ¿Qué diferencia hay entre esperar fetch y hacer un cálculo muy pesado dentro de un bucle?

Error frecuente: async no vuelve más rápido el servidor ni transforma automáticamente un cálculo en trabajo paralelo.

40. ¿Qué son los módulos, dependencias y variables de entorno?

Respuesta breve: Los módulos separan responsabilidades en archivos. Las dependencias son paquetes instalados. Las variables de entorno aportan configuración sin escribir secretos en el código.

Desde cero y ejemplo de DREAMS: CommonJS usa require/module.exports; los .mjs usan import/export. app/package.json define scripts y dependencias; app/pnpm-lock.yaml fija versiones. dotenv carga configuración local; Cloudflare entrega bindings al Worker. pnpm start usa Node 24 y el puerto local predeterminado es 8080. No se copian claves reales para estudiar.

Archivo y función: app/package.json; app/pnpm-lock.yaml; app/server/server.js: start; app/worker/official-client.mjs: create_official_database.

Repregunta posible: ¿Node, Express, pnpm y app/package.json cumplen la misma función?

Error frecuente: No subir .env reales, node_modules ni claves al frontend.

41. ¿Cómo se crea, autentica y elimina una cuenta?

Respuesta breve: Registro llama a Supabase Auth y crea un perfil customer; login valida credenciales. DREAMS no tiene endpoint para borrar usuarios ni para promover administradores.

Desde cero y ejemplo de DREAMS: El formulario envía POST /api/auth/register; registration_values valida y rechaza campos de rol. normal_registration usa signUp y registered_response entrega la sesión que realmente exista. Login usa signInWithPassword. Borrar una cuenta de prueba requiere gestión autorizada en Supabase Auth, revisando sesiones y relaciones; borrar profiles no borra auth.users. No se practica con Admin o clientes.

Archivo y función: app/server/customer-registration.js: registration_values, normal_registration; app/server/app.js: install_post_api_auth_login, install_post_api_auth_logout.

Repregunta posible: ¿El cliente puede enviar is_admin:true? ¿Cerrar sesión elimina la cuenta?

Error frecuente: No confundir perfil con identidad, borrar con logout, ni una cuenta nueva con un Admin.

42. ¿Cómo se modifica la base sin romper el proyecto?

Respuesta breve: Datos de prueba se modifican con la API autorizada. Cambios de estructura o funciones se hacen mediante una migración nueva, revisada y probada antes de aplicarla.

Desde cero y ejemplo de DREAMS: Una consulta como .from("products").update(datos).eq("id", id) modifica sólo las filas seleccionadas. SELECT lee, INSERT agrega, UPDATE cambia y DELETE borra; no son métodos HTTP. No se reescribe una migración aplicada. La RPC del carrito está modularizada por una migración nueva y su histórico se conserva. No se ejecutan seeds, resets ni migraciones reales para estudiar.

Archivo y función: app/supabase/migrations/20261105160613_modularize_account_cart.sql; app/server/app.js: install_put_api_admin_products_id; app/supabase/rollback/.

Repregunta posible: ¿Qué cambia entre editar un precio y agregar una columna? ¿Qué protege una transacción?

Error frecuente: No practicar modificaciones sobre datos comerciales ni confundir rollback con reiniciar la base.

43. ¿Cómo se conecta DREAMS con la base de datos?

Respuesta breve: El backend crea un cliente del SDK de Supabase con la URL del proyecto y una clave de servidor del mismo proyecto. El navegador pide datos a nuestra API; nunca recibe esa clave.

Desde cero y ejemplo de DREAMS: En local, dotenv carga la configuración del entorno y start comprueba SUPABASE_URL y SUPABASE_SECRET_KEY antes de crear el cliente. La configuración se toma como referencia de app/.env.example, sin completar ni subir valores reales. En la oficial esos valores llegan como bindings secretos/configuración al Worker y create_official_database crea el cliente. No se usan claves de capturas ni se corrigen claves borrando caracteres. Se conecta a la Data API por HTTPS mediante el SDK; no es una conexión SQL directa del navegador.

Ejemplo explicativo equivalente al patrón del servidor, con configuración ya disponible en un entorno de desarrollo aislado:

```
const { createClient } = require('@supabase/supabase-js');
const database = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SECRET_KEY,
  { auth: { persistSession: false, autoRefreshToken: false } }
);
```

El cliente compartido no mantiene la sesión de un usuario. Donde se requieren permisos por usuario se crea un cliente con su JWT, tomado de la sesión verificada. Una URL incorrecta, una clave de otro proyecto o un fallo de red no se arreglan cambiando la contraseña Admin. Para estudiar se usan fixtures o un proyecto de desarrollo autorizado; no se conecta un experimento a producción.

Archivo y función: app/server/server.js: start (línea 36); app/worker/official-client.mjs: create_official_database (línea 4); app/.env.example; app/server/app.js: configure_router.

Repregunta posible: ¿Por qué no se pone la clave de servidor dentro del JavaScript público? ¿Qué cambia entre el cliente compartido y uno con JWT?

Error frecuente: Supabase no es una librería que se instala para guardar los datos en el navegador; su SDK se comunica con servicios externos. No compartir claves ni imprimir configuración secreta para demostrar la conexión.

Inventario completo de endpoints

Se verificaron 46 combinaciones de método+ruta. HEAD se admite en lecturas según política y Express; no hay registros propios PATCH. OPTIONS no debe presentarse como una API CORS configurada. :id es un parámetro, no texto que se envía literalmente. Los filtros están en query; correo/contraseña de login están en body.

El archivo JSON adjunto conserva fuente, línea, propósito y permisos de cada combinación. Todas las escrituras pasan además por guard de origen y los límites aplicables. No significa que todo POST exija un header Origin: la política también considera Referer, contexto y cookies.

Habilitadas por la política de la oficial

Método	Ruta	Propósito	Permiso/requisito
GET	/api/admin/coupons	Listar cupones	Rol Admin verificado; cliente con JWT para datos
POST	/api/admin/coupons	Crear cupón	Rol Admin verificado; cliente con JWT para datos
DELETE	/api/admin/coupons/:id	Eliminar cupón	Rol Admin verificado; cliente con JWT para datos
PUT	/api/admin/coupons/:id	Editar cupón	Rol Admin verificado; cliente con JWT para datos
GET	/api/admin/inquiries	Consultar consultas	Rol Admin verificado; cliente con JWT para datos
GET	/api/admin/orders	Leer historial de pedidos, sin generar cobros	Rol Admin verificado; cliente con JWT para datos
GET	/api/admin/products	Listar productos para Admin	Rol Admin verificado; cliente con JWT para datos
POST	/api/admin/products	Crear producto	Rol Admin verificado; cliente con JWT para datos
DELETE	/api/admin/products/:id	Eliminar producto	Rol Admin verificado; cliente con JWT para datos
PUT	/api/admin/products/:id	Actualizar producto	Rol Admin verificado; cliente con JWT para datos
GET	/api/admin/reviews	Listar opiniones para moderación	Rol Admin verificado; cliente con JWT para datos
DELETE	/api/admin/reviews/:id	Eliminar opinión como Admin	Rol Admin verificado; cliente con JWT para datos
PUT	/api/admin/reviews/:id	Editar opinión como Admin	Rol Admin verificado; cliente con JWT para datos
GET	/api/admin/stats	Contar recursos	Rol Admin verificado; cliente con JWT para datos

Método	Ruta	Propósito	Permiso/requisito
GET	/api/admin/users	Consultar perfiles; no crea ni borra usuarios	Rol Admin verificado; cliente con JWT para datos
GET	/api/auth/config	Requisito de confirmación	Público; validaciones y límites según la operación
GET	/api/auth/google/config	Informar habilitación de Google	Público; validaciones y límites según la operación
POST	/api/auth/login	Verificar credenciales	Público; credenciales válidas y rate limiting
POST	/api/auth/logout	Salir del flujo actual	Sin guard login; si hay token intenta revocación local y borra cookies
GET	/api/auth/me	Consultar sesión	Devuelve usuario o null; 503 si no puede comprobar sesión
POST	/api/auth/register	Registrar customer y entregar sesión si existe	Público; entrada validada, sin rol elegido por el cliente
GET	/api/brands	Listar marcas	Público; validaciones y límites según la operación
GET	/api/cart/account	Leer carrito propio	Login; POST además valida scope/UUID de operación
POST	/api/cart/account	Aplicar operación al carrito propio	Login; POST además valida scope/UUID de operación
GET	/api/cart/config	Habilitación del carrito por cuenta	Público; validaciones y límites según la operación
POST	/api/cart/quote	Cotizar con precios y cupón del servidor	Público; validaciones y límites según la operación
GET	/api/checkout/config	Consultar modo de checkout activo	Público; validaciones y límites según la operación
GET	/api/config	Configuración pública y contacto	Público; validaciones y límites según la operación
GET	/api/health	Comprobar dependencias/API	Público; validaciones y límites según la operación
POST	/api/inquiries	Registrar consulta sin enviar WhatsApp	Público; validaciones y límites según la operación
POST	/api/presentation/payment	Revalidar ticket y emitir recibo académico	Modo académico activo; ticket/recibo y cookie de dueño según fase

Método	Ruta	Propósito	Permiso/requisito
POST	/api/presentation/result	Verificar recibo y devolver resultado	Modo académico activo; ticket/recibo y cookie de dueño según fase
POST	/api/presentation/session	Crear ticket académico	Modo académico activo; ticket/recibo y cookie de dueño según fase
GET	/api/products	Listar y filtrar catálogo	Público; validaciones y límites según la operación
GET	/api/products/:id	Consultar detalle	Público; validaciones y límites según la operación
GET	/api/reviews	Listar opiniones	Público; validaciones y límites según la operación
POST	/api/reviews	Publicar opinión propia	Login; autor tomado de sesión
DELETE	/api/reviews/:id	Eliminar opinión propia	Login, ID válido y dueño; JWT y filtro user_id

Preparadas: acceso Google no habilitado

Método	Ruta	Propósito	Permiso/requisito
POST	/api/auth/google	Preparar OAuth	Requiere proveedor/cliente OAuth y validaciones PKCE; callback valida contexto
GET	/api/auth/google/callback	Completar retorno OAuth	Requiere proveedor/cliente OAuth y validaciones PKCE; callback valida contexto

Históricas o preparadas bloqueadas en la oficial

Método	Ruta	Propósito	Permiso/requisito
POST	/api/checkout/confirm	Confirmación histórica de pago	En backend genérico requiere login y configuración; oficial bloquea
POST	/api/checkout/demo-payment	Pago demo histórico	En backend genérico requiere login y configuración; oficial bloquea
POST	/api/checkout/session	Checkout comercial/demo histórico	En backend genérico requiere login y configuración; oficial bloquea
POST	/api/coupons/validate	Validación independiente histórica de cupón	Público; validaciones y límites según la operación

Método	Ruta	Propósito	Permiso/requisito
GET	/api/orders/:id	Consultar un pedido histórico del cliente	En backend genérico requiere login y comprobación de dueño; oficial bloquea
POST	/api/payments/webhook	Recibir notificación histórica del proveedor	En backend genérico verifica proveedor/firma según flujo; oficial bloquea

GET /api/auth/google/config sí informa el estado. Que una ruta esté en una allowlist no garantiza que su proveedor esté configurado ni que el usuario tenga permisos. En la oficial, el cupón se valida por /api/cart/quote, no por la ruta histórica independiente.

Las páginas públicas son /, /catalogo.html, /producto.html, /nosotros.html, /cuenta.html, /carrito.html, /checkout.html y /checkout-resultado.html; también existe 404.html. No son endpoints de datos. /admin y /admin.html son vistas protegidas; /js/admin-console-v3.js es código cliente, no permiso para usar la API. /favoritos y /favoritos.html son redirecciones del servidor genérico. Imágenes/CSS/JS son recursos estáticos, no 46 APIs adicionales.

Funciones reales: entradas, proceso, salida y errores

F01. Catálogo y filtros: apply_catalog_filters

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Normaliza parámetros, actualiza historial/tema y espera load_catalog. Promise sin datos de retorno de negocio; cambia URL, contador, clases y contenido.

Ubicación: app/public/js/catalogo.js:84; apply_catalog_filters.

Entradas: Estado aplicado y opción sync_url.

Proceso: Normaliza parámetros, actualiza historial/tema y espera load_catalog.

Retorno y efectos: Promise sin datos de retorno de negocio; cambia URL, contador, clases y contenido.

Errores: load_catalog protege respuestas atrasadas con request_id y muestra error recuperable.

Ejercicio local seguro: Probar dos filtros con respuestas locales en orden inverso; gana el último.

Repregunta/error frecuente: No decir que cambiar la URL modifica productos.

F02. Consulta del catálogo: load_catalog

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Muestra loading, hace GET /api/products, valida array y acepta sólo la respuesta vigente. Actualiza catalog_products y renderiza; en finally ajusta aria-busy si sigue vigente.

Ubicación: app/public/js/catalogo.js:90; load_catalog.

Entradas: Filtros actuales e ID de la petición.

Proceso: Muestra loading, hace GET /api/products, valida array y acepta sólo la respuesta vigente.

Retorno y efectos: Actualiza catalog_products y renderiza; en finally ajusta aria-busy si sigue vigente.

Errores: HTTP inválido o fallo de red muestra catalog_error; una respuesta vieja no pisa la nueva.

Ejercicio local seguro: Explicar por qué request_id se compara antes de renderizar.

Repregunta/error frecuente: await no evita por sí solo respuestas fuera de orden.

F03. Registro: normal_registration

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Llama signUp; guarda perfil customer; consulta confirmación si no hay sesión. Respuesta HTTP con sesión real si existe; crea identidad/perfil, no Admin.

Ubicación: app/server/customer-registration.js:32; normal_registration.

Entradas: Contexto de servicios, valores ya validados y response.

Proceso: Llama signUp; guarda perfil customer; consulta confirmación si no hay sesión.

Retorno y efectos: Respuesta HTTP con sesión real si existe; crea identidad/perfil, no Admin.

Errores: registration_failure traduce el error; el proveedor puede devolver usuario sin sesión y no se inventa login.

Ejercicio local seguro: Con fixture, simular signUp sin session y predecir la respuesta.

Repregunta/error frecuente: No confirmar cuentas administrativamente como atajo.

F04. Login: install_post_api_auth_login

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Normaliza/valida correo, llama signInWithPassword, obtiene perfil y escribe cookies. Registra POST /api/auth/login; el handler devuelve JSON con usuario y efectos de sesión.

Ubicación: app/server/app.js:472; install_post_api_auth_login.

Entradas: Contexto; el handler recibe body.email/body.password.

Proceso: Normaliza/valida correo, llama signInWithPassword, obtiene perfil y escribe cookies.

Retorno y efectos: Registra POST /api/auth/login; el handler devuelve JSON con usuario y efectos de sesión.

Errores: 400 por entrada inválida, error de credenciales y 503 por indisponibilidad se distinguen.

Ejercicio local seguro: Localizar la diferencia entre contraseña incorrecta y proveedor caído.

Repregunta/error frecuente: POST no cifra ni obliga a que jamás se registren datos.

F05. Permisos Admin: configure_router

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Crea login, admin, route y admin_database basado en token de sesión. Agrega helpers al contexto y registra rutas a través de route. admin llama next o responde 403.

Ubicación: app/server/app.js:367; configure_router.

Entradas: Aplicación, cliente Auth y envoltorio async.

Proceso: Crea login, admin, route y admin_database basado en token de sesión.

Retorno y efectos: Agrega helpers al contexto y registra rutas a través de route. admin llama next o responde 403.

Errores: Sin rol comprobado no se ejecuta la operación; no se toma is_admin del JSON.

Ejercicio local seguro: Señalar cómo una petición anónima a un endpoint Admin queda rechazada.

Repregunta/error frecuente: Ocultar un enlace no es autorización.

F06. Guardar producto e imagen: commit_product

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Bloquea doble envío, revisa alta incierta, escribe, confirma todos los campos con readback y actualiza listados/conteos. Promise; sólo anuncia creado/actualizado después de confirmar persistencia. No sube una imagen: guarda su URL.

Ubicación: app/public/js/admin.js:276; commit_product.

Entradas: ID opcional y datos del formulario validados.

Proceso: Bloquea doble envío, revisa alta incierta, escribe, confirma todos los campos con readback y actualiza listados/conteos.

Retorno y efectos: Promise; sólo anuncia creado/actualizado después de confirmar persistencia. No sube una imagen: guarda su URL.

Errores: catch conserva información y muestra error; finally restaura controles. La recuperación evita duplicar un alta incierta.

Ejercicio local seguro: Revisar con fixtures un POST exitoso seguido de lectura fallida.

Repregunta/error frecuente: Un 200 de escritura o una preview de imagen no prueba persistencia completa.

F07. Confirmar persistencia: confirm_product_persistence

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Verifica ID válido, relee /api/admin/products y compara cada campo normalizado. Resuelve sin valor si coincide; rechaza con error si no puede confirmar. Sólo lee.

Ubicación: app/public/js/admin.js:246; confirm_product_persistence.

Entradas: Producto devuelto y valores esperados.

Proceso: Verifica ID válido, relee /api/admin/products y compara cada campo normalizado.

Retorno y efectos: Resuelve sin valor si coincide; rechaza con error si no puede confirmar. Sólo lee.

Errores: No encuentra el recurso o difieren campos: lanza error y el guardado no se anuncia como confirmado.

Ejercicio local seguro: Predecir el resultado si la URL de imagen leída es distinta.

Repregunta/error frecuente: No mostrar Guardado antes de esta comprobación.

F08. Buscador Admin: filter_products

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: normalize_search quita marcas diacríticas; filter combina stock válido y coincidencia en nombre o marca con some/includes. Devuelve lista filtrada; no escribe en DB ni borra filas. Sin filtros puede devolver la misma lista.

Ubicación: app/public/js/admin-product-search.js:3; filter_products.

Entradas: Array de productos, texto y low_only.

Proceso: normalize_search quita marcas diacríticas; filter combina stock válido y coincidencia en nombre o marca con some/includes.

Retorno y efectos: Devuelve lista filtrada; no escribe en DB ni borra filas. Sin filtros puede devolver la misma lista.

Errores: No convierte stock null en cero; el llamador aporta un array válido.

Ejercicio local seguro: Probar normalize_search con ÁMBAR y low_stock con 0, 1, 2, 3, null y "2".

Repregunta/error frecuente: Un texto "2" no es stock numérico válido.

F09. Carrito por cuenta: async write

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Valida forma/UUID, compara scope con HMAC de la cuenta, llama dreams_cart_operation y luego hydrate. Devuelve JSON con revisión e items/precios actuales; la RPC guarda carrito y registro idempotente en transacción.

Ubicación: app/server/account-cart.js:43; async write.

Entradas: req.body con scope, operation_id y operation; usuario restaurado en req.user.

Proceso: Valida forma/UUID, compara scope con HMAC de la cuenta, llama dreams_cart_operation y luego hydrate.

Retorno y efectos: Devuelve JSON con revisión e items/precios actuales; la RPC guarda carrito y registro idempotente en transacción.

Errores: 503 si no está listo; 400 entrada inválida; 409 por cambio de contexto/límites. Otros errores van al manejador central.

Ejercicio local seguro: Seguir el caso de operación de la cuenta A enviada con sesión B usando fixtures.

Repregunta/error frecuente: El UUID escrito por el cliente no autoriza el dueño.

F10. Opinión propia: delete_own_review

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Valida ID, usa cliente con JWT y aplica id + user_id en delete. JSON de éxito; elimina sólo una fila propia que realmente coincida.

Ubicación: app/server/reviews.js:33; delete_own_review.

Entradas: params.id y sesión verificada; cuerpo vacío.

Proceso: Valida ID, usa cliente con JWT y aplica id + user_id en delete.

Retorno y efectos: JSON de éxito; elimina sólo una fila propia que realmente coincida.

Errores: 400 entrada inválida; 404 si no hay fila propia; error de proveedor se deriva.

Ejercicio local seguro: Leer la prueba de intento de borrar una opinión ajena.

Repregunta/error frecuente: No usar la clave privilegiada para borrar cualquier opinión desde la ruta customer.

F11. Checkout local: presentation_submit

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Valida nombre/formato, bloquea envío, comprueba carrito, recupera intención y espera presentation_complete. Promise; guarda sólo intención/ticket/recibo, limpia formulario y navega al resultado tras confirmar.

Ubicación: app/public/js/presentation.js:80; presentation_submit.

Entradas: Botón, estado, formulario y snapshot inicial.

Proceso: Valida nombre/formato, bloquea envío, comprueba carrito, recupera intención y espera presentation_complete.

Retorno y efectos: Promise; guarda sólo intención/ticket/recibo, limpia formulario y navega al resultado tras confirmar.

Errores: Un error muestra Reintentar y conserva referencia/carrito; no trata datos de tarjeta como payload.

Ejercicio local seguro: Seguir un fallo simulado de red y comparar el intent_id del reintento.

Repregunta/error frecuente: El nombre no se envía como dato de pago; formato local no es autorización bancaria.

F12. Checkout del servidor: install_presentation

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Instala session/payment/result; valida campos, dueño y vencimiento, revalida cupón/importes y firma ticket o recibo. Respuestas JSON firmadas; no inserta orders ni descuenta stock ni consume usos de una compra pagada.

Ubicación: app/server/presentation-checkout.js:31; install_presentation.

Entradas: Servicios de firma/lectura, quote_cart, configuración y registrador de rutas.

Proceso: Instala session/payment/result; valida campos, dueño y vencimiento, revalida cupón/importes y firma ticket o recibo.

Retorno y efectos: Respuestas JSON firmadas; no inserta orders ni descuenta stock ni consume usos de una compra pagada.

Errores: 404 si modo no listo, 400 token/cuerpo inválido, 409 por cambios de importes/cupón.

Ejercicio local seguro: Mostrar por qué un total o ticket manipulado se rechaza en fixtures.

Repregunta/error frecuente: Firmado no quiere decir cifrado ni cobrado.

F13. Control de origen: mutation_origin_guard

Pregunta probable: ¿Qué entra, qué hace y qué devuelve esta función?

Para decir: Sólo actúa sobre escrituras. Considera Origin/Referer y, sin ellos, contexto cross-site y cookies; normaliza y compara allowlist. Devuelve middleware; permite next o corta con 403. No concede rol ni configura ACAO.

Ubicación: app/server/app.js:47; mutation_origin_guard.

Entradas: Orígenes configurados; luego request/response/next.

Proceso: Sólo actúa sobre escrituras. Considera Origin/Referer y, sin ellos, contexto cross-site y cookies; normaliza y compara allowlist.

Retorno y efectos: Devuelve middleware; permite next o corta con 403. No concede rol ni configura ACAO.

Errores: Origen inválido/no admitido o petición inducida sin señales apropiadas puede ser rechazada.

Ejercicio local seguro: Leer con fixture una escritura desde un origen extraño con cookie.

Repregunta/error frecuente: Esto no es la biblioteca cors ni reemplaza autenticación.

Práctica sin tocar datos reales

Los ejercicios son propuestas, no actividades ya realizadas por los integrantes. Usar una copia local, fixtures o una DB aislada. No ejecutar scripts de alta de Admin, reparadores de Secrets, seeds, resets, borrados reales ni publicación para estudiar.

```
cd app
pnpm run check
pnpm run check:functions
node --test test/home-catalog-copy.test.js
node --test test/account-cart.test.js test/review-ownership.test.js
node --test test/admin-persistence.test.js
```

Los nombres de los tests anteriores existen en la base revisada. No se ejecutaron nuevamente en esta revisión documental. Para arrancar conectado hace falta un Supabase de desarrollo correctamente configurado; no se incluyen ni solicitan secretos en esta guía.

Guion de un minuto: explicar qué muestra la interfaz, seguir una llamada fetch, ubicar el endpoint/método, mostrar validación/permisos, indicar qué servicio consulta y cómo vuelve la respuesta. Si no podés justificar una línea, volver a su archivo en lugar de inventar una explicación.

Fuentes, correcciones y límites

La guía adjunta se contrastó con main 60451ca: no hay cambio funcional posterior que reconciliar en esa base. Se retira de la publicación el texto de los audios y la identificación del participante. Se completan configuraciones, rutas Google preparadas e históricos para cubrir las 46 combinaciones API; la tabla parcial del adjunto no era el inventario completo.

Express se describe como framework web; no se atribuye su elección a los integrantes. La revocación de logout se describe como intento de alcance local más vencimiento de cookies, no como garantía de invalidar todo token. No se afirma que DREAMS implemente bcrypt ni tenga una dependencia cors.

La documentación oficial de Supabase sobre seguridad de contraseñas, consultada el 10/11/2026, declara bcrypt con salt aleatorio. Esa es una afirmación del proveedor; no se consultaron contraseñas ni hashes de producción ni se afirmó un costo concreto.

- Proveedor: https://supabase.com/docs/guides/auth/password-security
- Código revisado: https://github.com/facumartea/dreams/tree/60451cabe5ff9b78f97777d64120e5ff9a3238b2
- Inventario verificado de endpoints.
- Registro consolidado parcial de prompts.
- Limpieza y archivos preservados.
- Índice de entrega.

Esta tanda no ejecuta nuevamente QA de navegador, Auth, Admin, base remota ni despliegues. Reutiliza evidencia anterior con su fecha/código y valida documentos, rutas y referencias estáticamente. El CI obligatorio de GitHub puede ejecutar sus checks automáticos al abrir/integrar la PR. Participación, preentregas y defensa individual necesitan evidencia humana; no se garantizan notas.